

SAP BTP as the Foundation for Partner SaaS Solutions

Architectural choices, development approaches, and operational lessons from taking five solutions to the SAP Store

Geert-Jan Klaps
Partner, Head of Engineering & SAP Mentor



Agenda



Why Multi-Tenancy on SAP BTP?



Architecture & Design Decisions



Lessons Learned



Real-World Example



Wrap-up & Q&A

Why Multi-Tenancy on SAP BTP?

The SaaS Opportunity on SAP BTP



Cost Efficiency at Scale

Multi-tenancy dramatically reduces total cost of ownership. Shared infrastructure, single codebase, centralized operations across all customers.



Faster Innovation

Ship once and every tenant benefits: one codebase, continuous delivery, and platform services that remove the plumbing.



SAP Partner Ecosystem

SAP BTP provides architecture patterns, development practices, and go-to-market support through SAP PartnerEdge Build and the SAP Store.



Built-In Platform Services

Leverage CAP, HANA Cloud, Identity Authentication, SaaS Provisioning, and Cloud Foundry/Kyma for enterprise-grade SaaS foundations.



SAP Store Distribution

Reach SAP customers globally via SAP Store. Streamlined onboarding, commercial models, and integration into SAP landscapes.

Our SaaS Portfolio on SAP BTP

Find us on SAP Store



GoBeyond AI DataBridge

Connect any agent to any backend system, zero coding required.

>10 customers

✓ Passed the SAP interoperability review

GoBeyond Supplier Invoicing

Automated invoice processing with OCR, Peppol e-invoicing and ERP posting.

>10 customers

✓ Certified Built with SAP Business AI

GoBeyond Supplier Confirmations

AI-powered processing of supplier order confirmations directly in SAP Cloud ERP.

>10 customers

✓ Certified Built with SAP Business AI

GoBeyond Integration Monitoring

Proactive error detection across SAP Cloud Integration flows.

>10 customers

GoBeyond Customer Portal

Self-service portal for order management, document access and real-time process visibility.

First customers

Every one of these solutions is provisioned and live within one day.

Architecture & Design Decisions

Multi-Tenant Architecture on SAP BTP



Key BTP services powering multi-tenant SaaS

- SAP SaaS Provisioning Service for tenant lifecycle management (subscribe, unsubscribe, upgrade)
- SAP HANA Cloud with tenant-isolated HDI containers for strict data separation
- SAP Cloud Identity Services (IAS/IPS) for enterprise-grade authentication and user federation
- SAP Cloud Application Programming Model (CAP) with built-in @sap/cds-mtxs multi-tenancy

Core Abstractions That Matter



Tenancy Models & Boundaries

Shared app instances with tenant-specific DB containers. CAP abstracts tenant isolation, schema evolution, and provisioning flows.



Identity & Authorization

XSUAA / IAS for OAuth-based auth. Role collections per tenant, trust configuration, and principal propagation to backend systems.



Data & Service Contracts

CDS-based domain models with safe extensibility via feature toggles. Stable APIs with OData/REST contracts for tenant customization.



Lifecycle Agility

Blue-green deployments, zero-downtime upgrades, tenant-aware schema migration. Automated scaling with CF Application Autoscaler.

Lessons Learned

Development Best Practices



TypeScript for SAP Development

- Enforces type-safety, fewer runtime bugs
- Improves code maintainability in larger projects
- Enhanced productivity: autocompletion, navigation, refactoring
- Supported in both SAP CAP and SAPUI5



Provide Sufficient Logging

- Debugging in production is not an option
- Use the SAP CAP Logging Facade consistently
- Always include Tenant ID, Subdomain, and User context
- SAP Cloud Logging dashboards for troubleshooting

Extensibility & Resilience



"Pluginify" Your Solutions

- Leverage standard SAP CAP plugins
- Create plugins for cross-solution reusable functionality
- Reduces long-term maintenance effort significantly
- Ours so far: feature flags, user management, DMS integration, background jobs, MCP



Build with Resilience in Mind

- SAP provides 99.9% SLA, but 0.1% still means ~45 min/month
- Design for potential disruptions in every service dependency
- Example: DMS outage → queue mails for reprocessing later
- Use circuit breakers and graceful degradation patterns

Disaster Recovery on SAP BTP

Shared Responsibility Model

SAP provides disaster recovery for all BTP services, but applications on Cloud Foundry and HANA Cloud DB follow a shared responsibility model.

In-Metro Disaster Recovery (AWS & Azure)

- Recovery Point Objective (RPO): max 5 minutes of data loss
- Recovery Time Objective (RTO): full-service restoration within 2 hours
- Use Application Auto Scaler for Cloud Foundry applications
- HANA Cloud must be replicated to a secondary site

RPO

5 min

max data loss

RTO

2 hrs

full restoration

Real-World Example

GoBeyond Supplier Invoicing

Background jobs, factored out into a CAP plugin of our own

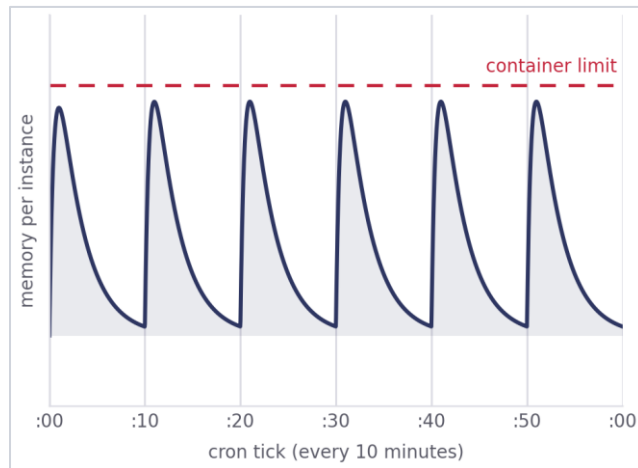
Before: One Cron, Every Tenant, the Same Minute

Driven by the SAP BTP Job Scheduling Service

Delta sync, mail intake and OCR polling all carried the same ten-minute cron, and customizing sync ran at midnight. Cron fires on fixed wall-clock times, so every tenant started together.

Why That Showed Up as Memory

- The scheduler wanted a fast 202, so the work ran detached behind it
- No queue and no backpressure, so a slow run overlapped the next one
- Every job ran inside -srv, beside live UI traffic; no worker existed
- N tenants syncing at once is N times the heap, in one container



Illustrative: the peaks are the cron boundary, not a measurement

Moving the Jobs Out of the Application Service



One build artefact, two roles, separated by configuration alone

- CDS_APPID partitions the shared cds.outbox.Messages table, so each module only sees its own tasks
- No UI action enqueues anything: buttons record intent, and the worker's reconciler turns it into a real task
- /rest/jobs/* routes through the approuter to the worker; a POST there runs the job in the receiving process
- Net effect: a full sync can no longer execute inside a 768 MB UI instance, competing with live user traffic

Inside Our Own Jobs Plugin



Configuration, Not Code

Three CDS lines and a JSON block. The plugin finds its own entity by annotation and registers seeding, decoration and the bound actions at served.



Run Locks That Age Out

A conditional UPDATE per tenant job, committed in its own transaction so others can see it, and expiring on lockStaleAfterMs so a crash cannot wedge the job forever.



A Run Log Per Run

Every run is bracketed with a start and a finish entry. A failed run is re-written from a detached transaction, because the queue rolls the handler's back.



One Verdict, Three Readers

Is the schedule live? Is a run in hand? Is a one-off spoken for? All three derive from a single verdict(); two judgements over one row is how jobs go silently dark.

What the Tenant Administrator Sees

<  Tenant Configuration ▾

GK

Tenant Configuration

Edit  ▾

Jobs

Jobs (5)

Search Q Reset    ▾

☐ Job Type	Job Status	Next Run Scheduled	Last Run Started	Last Run Completed	Last Run Status	Active				
☐ Customizing synchronization	✓ COMPLETED	Sep 10, 2026, 3:04:31 PM	Sep 9, 2026, 3:04:14 PM	Sep 9, 2026, 3:04:31 PM	✓ COMPLETED	Yes			>	
☐ Full synchronization	✓ COMPLETED		Sep 9, 2026, 6:08:58 PM	Sep 9, 2026, 6:09:48 PM	✓ COMPLETED	No			>	
☐ OCR result	✓ COMPLETED	Sep 9, 2026, 6:19:49 PM	Sep 9, 2026, 6:09:49 PM	Sep 9, 2026, 6:09:49 PM	✓ COMPLETED	Yes			>	
☐ Delta synchronization	✓ COMPLETED	Sep 9, 2026, 6:19:58 PM	Sep 9, 2026, 6:09:49 PM	Sep 9, 2026, 6:09:58 PM	✓ COMPLETED	Yes			>	
☐ Inbound processing	✓ COMPLETED		Sep 1, 2026, 9:55:39 AM	Sep 1, 2026, 9:55:39 AM	✓ COMPLETED	No			>	

- Five jobs, seeded per tenant on first read; the consuming service writes no handler for any of this
- Schedule, Pause, Run Now and Reset are the plugin's own bound actions, enabled per row
- RUNNING is read from the queue, because the run's own history row commits only when the run ends

Recovering an Interrupted Run

A deployment killed the 17:40 run: the history flags it, the run log explains it

Started 	Finished	Outcome	Message	
Sep 9, 2026, 6:09:49 PM	Sep 9, 2026, 6:09:58 PM	✓ COMPLETED		>
Sep 9, 2026, 5:58:49 PM	Sep 9, 2026, 5:59:01 PM	✓ COMPLETED		>
Sep 9, 2026, 5:40:21 PM	Sep 9, 2026, 5:56:58 PM	⚠ ABANDONED	Interrupted before it could finish	>
Sep 9, 2026, 5:30:10 PM	Sep 9, 2026, 5:30:21 PM	✓ COMPLETED		>
Sep 9, 2026, 5:20:01 PM	Sep 9, 2026, 5:20:10 PM	✓ COMPLETED		>

Time 	Severity	Code	Message
Sep 9, 2026, 5:56:58 PM	⚠ WARNING	ABANDONED	The process running this job ended before it could finish — most likely an instance restart or a deployment. The queue has held task job-delta-sync as 'processing' since 2026-09-09T15:40:21.440Z and would never have selected it again: a task interrupted on its first attempt has no lastAttemptTimestamp, so the queue's own reclaim window cannot apply to it. The schedule has been re-created.

Load Distribution: Slots and Backlog-Driven Scaling

Concurrency Slots Bound the Container

The run lock covers one job type in one tenant. Two tenants' full syncs still shared an instance, and two large in-memory syncs in one container is an out-of-memory restart. Jobs naming the same slot now cannot run together in one process.

Scale on Required Instances, Not on CPU

- cpu is inverted: I/O-bound jobs sit at 20% and trip $\text{cpu} < 30 \rightarrow -1$
- memoryutil ratchets: scale-out fires, scale-in effectively never can
- So the plugin submits $\max(\text{ceil}(\text{dueLight} / \text{chunkSize}), \text{dueHeavy})$ instead
- Dead and abandoned tasks excluded, or one row pins the fleet at max

SLOT

1

heavy job per process

CHUNK

5

light tasks per instance

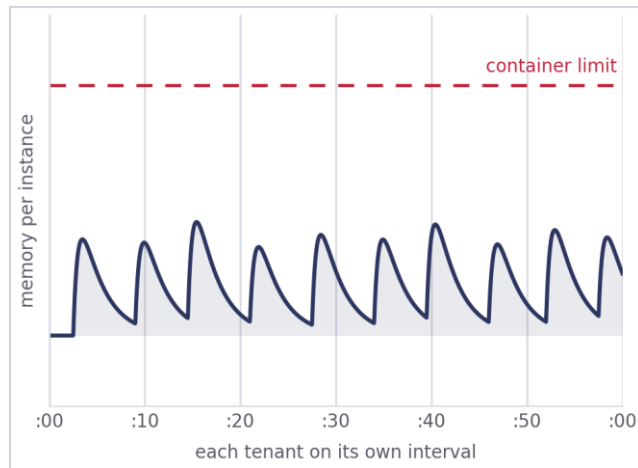
After: Every Tenant on Its Own Clock

Driven by CAP's Transactional Event Queue

The every interval is a post-execution delay: the next occurrence is created only once the current one succeeds, so a run can never overlap itself. Tenants drift apart on their own completion times, and a hashed offset spreads even their first runs.

What Became of the Peaks

- No shared boundary, so the fleet never wakes up all at once
- One heavy slot per process: two full syncs cannot share a container
- Jobs moved to the worker, out of the instances serving the UI
- Instances follow the queue backlog, so capacity tracks real work



Illustrative: the same hour, anchored to each tenant's own finish

Before and After, in Containers



Before: Everything Inside -srv

- -srv at 1024 MB, autoscaled from 3 to 5 instances
- Jobs ran in those same containers, beside UI traffic
- Scaled on cpu and memoryutil, which track neither
- Always-on floor of 3 GB, whatever the queue did



After: A Worker of Its Own

- -srv at 768 MB, autoscaled from 1 to 3 instances
- Worker at 1024 MB, 1 to 5, driven by queue backlog
- Always-on floor of 1.75 GB, down from 3 GB
- Five workers, but only when work is actually due

Module sizing and autoscaler rules from mta.yaml, before the split and today.

Wrap-up

Key Takeaways

1 Leverage SAP BTP best practices: CAP, Fiori Elements, SAPUI5 for scalable multi-tenant SaaS

2 Think carefully about tenancy boundaries, identity models, and data contracts early in the design

3 "Pluginify" your solutions: our jobs plugin turned tenant scheduling into configuration, not code

4 Provide sufficient logging and build with resilience in mind – production debugging is not an option

5 Don't forget the shared responsibility model: HANA Cloud replication + Application Auto Scaler for DR

Questions?

Thank you for attending!

Geert-Jan Klaps

geert-jan.klaps@thevaluechain.eu



SAP Store



LinkedIn